

AIOS-指导AI 工程方向

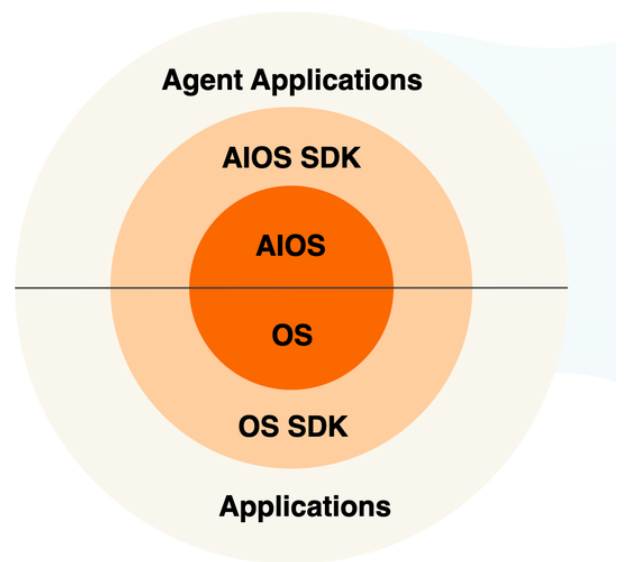
1. 为什么要了解AIOS 项目

1. AIOS是智能体时代，非常重要的概念、设计思路；
2. 研究+实现，有论文，有实践，可以了解AIOS的全貌
3. 优秀的工程架构，异步+扩展性

2. 什么是AIOS(Agent-Integrated OS)

AIOS: LLM Agent Operating System

AIOS is a large language model (LLM) agent operating system, which embeds large language model into the operating system as the brain of the OS. AIOS is designed to address problems such as scheduling, context switch, memory management, tool management and access management during the development and deployment of LLM-based agents, for a better ecosystem among agent developers and users.

[Contribute](#)[Paper](#)

最大差异性

1. 用户交互GUI
2. APP与外界交互
3. 核心原理&论文

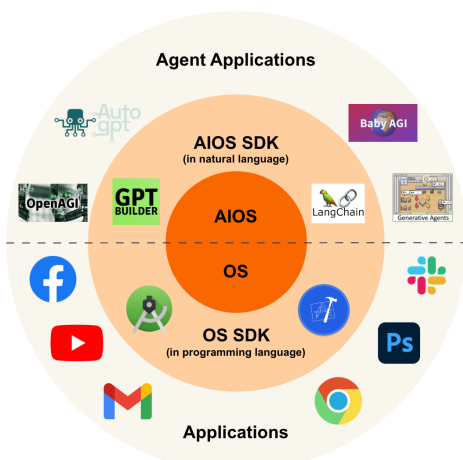
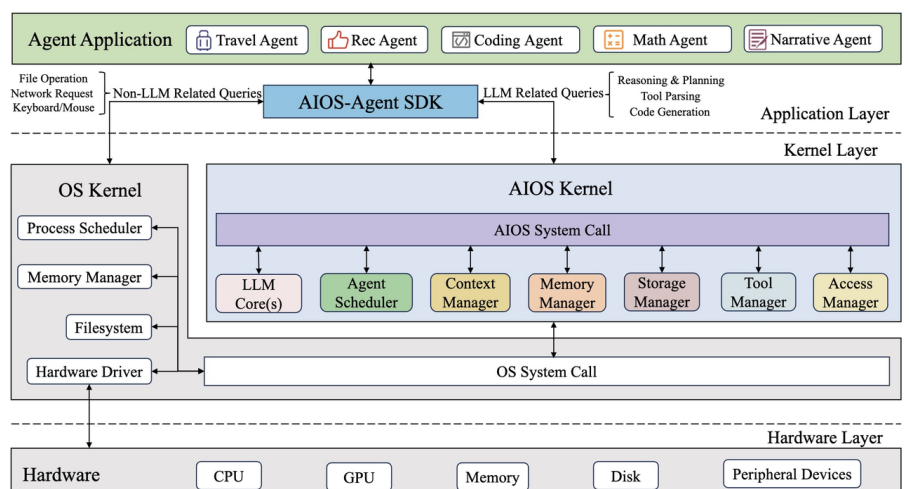


Figure 1: OS-APP ecosystem vs. AIOS-Agent ecosystem.



3.1 LLM as OS, Agents as Apps: Envisioning AIOS, Agents and the AIOS-Agent Ecosystem

该论文设想了一个革命性的AIOS-Agent 生态系统，其中大型语言模型（LLM）作为智能操作系统（AIOS），类似于传统操作系统的内核，而基于 LLM 的 AI 代理应用（Agents/AAPs）则作为应用程序。

论文通过将 AIOS 与传统操作系统进行类比，阐述了其概念框架，包括 LLM 比作 OS 内核、上下文窗口比作内存等。同时介绍了新的 AIOS-Agent 生态系统，用户可通过自然语言轻松编程 AAPs，还探讨了代理应用的范围，最后借鉴传统 OS-APP 生态系统提出了该生态系统的发展路线图，旨在指导未来的研发，这标志着从传统 OS-APP 生态系统的范式转变。

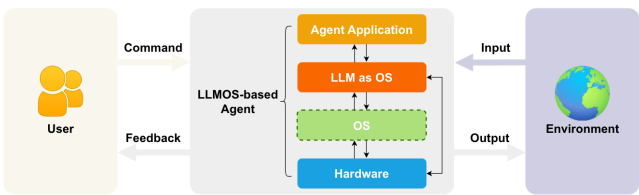
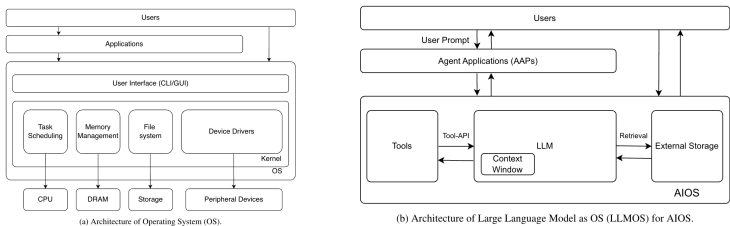
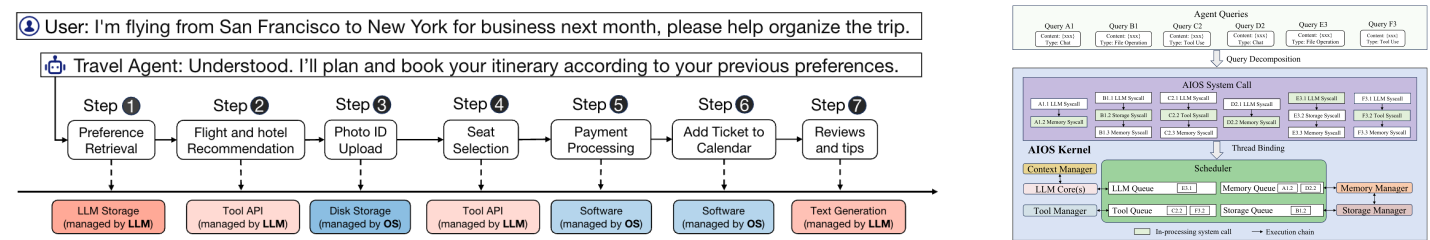


Figure 4: An illustration of LLMOS-based AI Agent.

3.2 AIOS: LLM Agent Operating System

本文提出AIOS（基于 LLM 的 AI 代理操作系统）架构，旨在解决基于 LLM 的智能代理在部署中面临的资源管理挑战，如资源分配低效、缺乏合理调度机制等。该架构将资源和 LLM 特定服务从代理应用中隔离到 AIOS 内核，内核提供调度、上下文管理等基础服务及对 LLM 和外部工具等资源的高效管理，还包含 AIOS-Agent SDK 供调用内核功能。实验表明，使用 AIOS 可使不同代理框架构建的代理执行速度提升高达2.1 倍，源代码已公开。



3.3 From Commands to Prompts: LLM-based Semantic File System for AIOS

提出了一种基于大语言模型（LLMs）的语义文件系统（LSFS），旨在解决传统文件系统依赖精确命令和手动导航复杂文件夹层级的瓶颈。该系统通过自然语言提示实现用户或代理与文件的交互，在宏观层面开发了涵盖语义文件检索、更新监控与总结、语义回滚等功能的综合 API 集，微观层面通过构建语义索引存储文件，并借助向量数据库设计实现了不同语义操作的系统调用（如 CRUD、分组、连接等）。

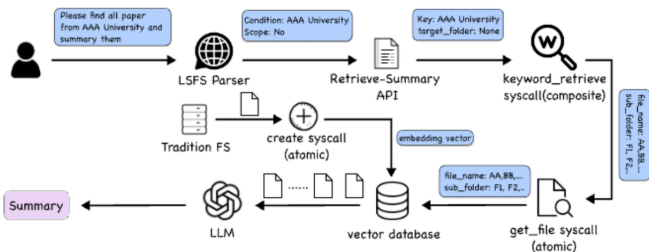


Figure 1: A fine-grained example of the pipeline of changing file of traditional system and our LSFS.

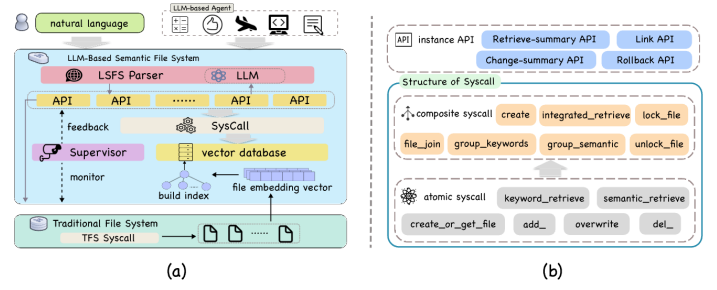
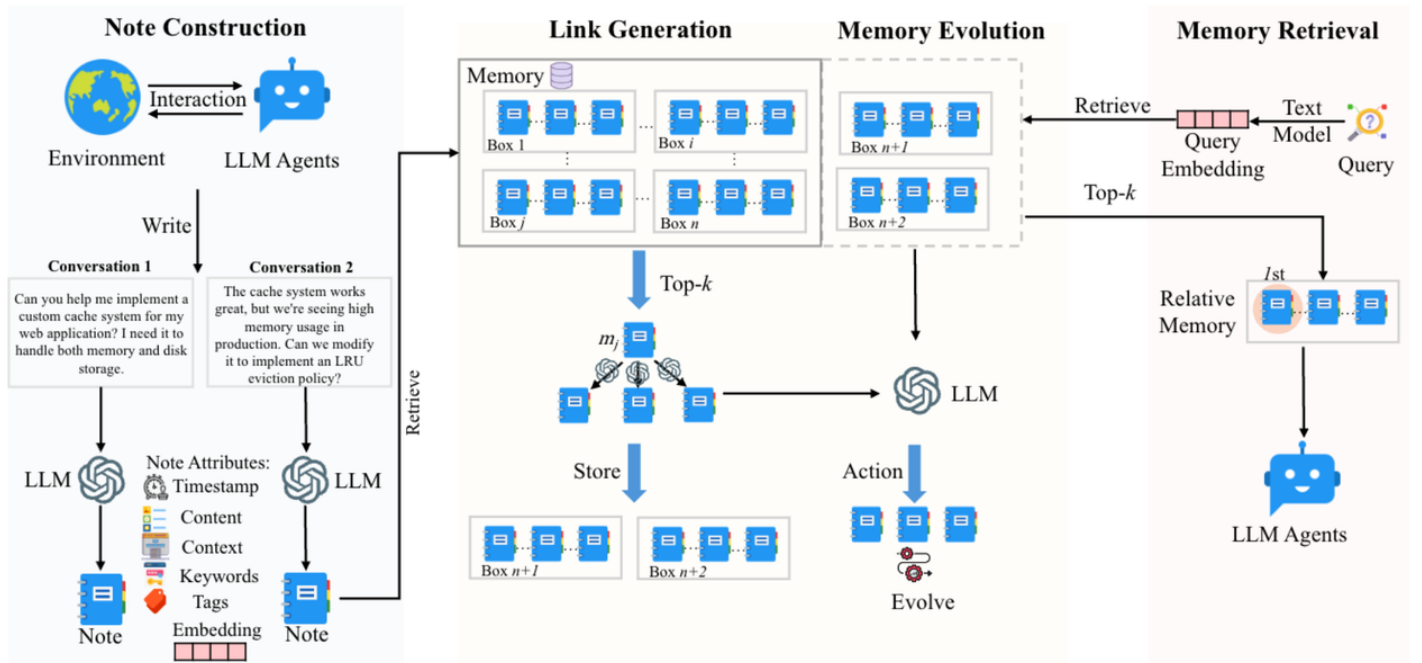


Figure 2: (a) provides an overview of the LSFS architecture, and (b) shows the internal APIs and syscalls in LSFS.

3.4 A-MEM: Agentic Memory for LLM Agents

本文提出A-MEM这一新型LLM 代理的智能记忆系统，其借鉴Zettelkasten 方法基本原理，通过动态索引和链接创建互联知识网络。新记忆加入时会生成含上下文描述、关键词、标签等结构化属性的综合笔记，系统分析历史记忆建立有意义联系，还能实现记忆进化，即新记忆整合触发历史记忆的上下文表示和属性更新，使记忆网络持续优化理解。该方法结合 Zettelkasten 的结构化组织原则与代理驱动决策的灵活性，经六个基础模型的实证实验，表现优于现有 SOTA 基线，相关源代码可通过提供的 URL 获取。



3.5 Cerebrum (AIOS SDK): A Platform for Agent Development, Deployment, Distribution, and Discovery

论文介绍了Cerebrum（AIOS SDK），这是一个用于智能体开发、部署、分发和发现的平台，旨在解决基于 LLM 的自主智能体领域缺乏标准化工具的问题。其核心包含三个关键组件：具有模块化四层架构（涵盖 LLM、内存、存储和工具管理）的综合 SDK、社区驱动的用于共享和发现智能体的 Agent Hub（含版本控制和依赖管理）、以及用于测试和评估智能体的交互式网页界面。该平台通过实现包括 Chain of Thought（CoT）、ReAct 和工具使用智能体等多种智能体架构展示了有效性，已被 2025 年北美计算语言学协会年会（NAACL）的系统演示轨道接受。

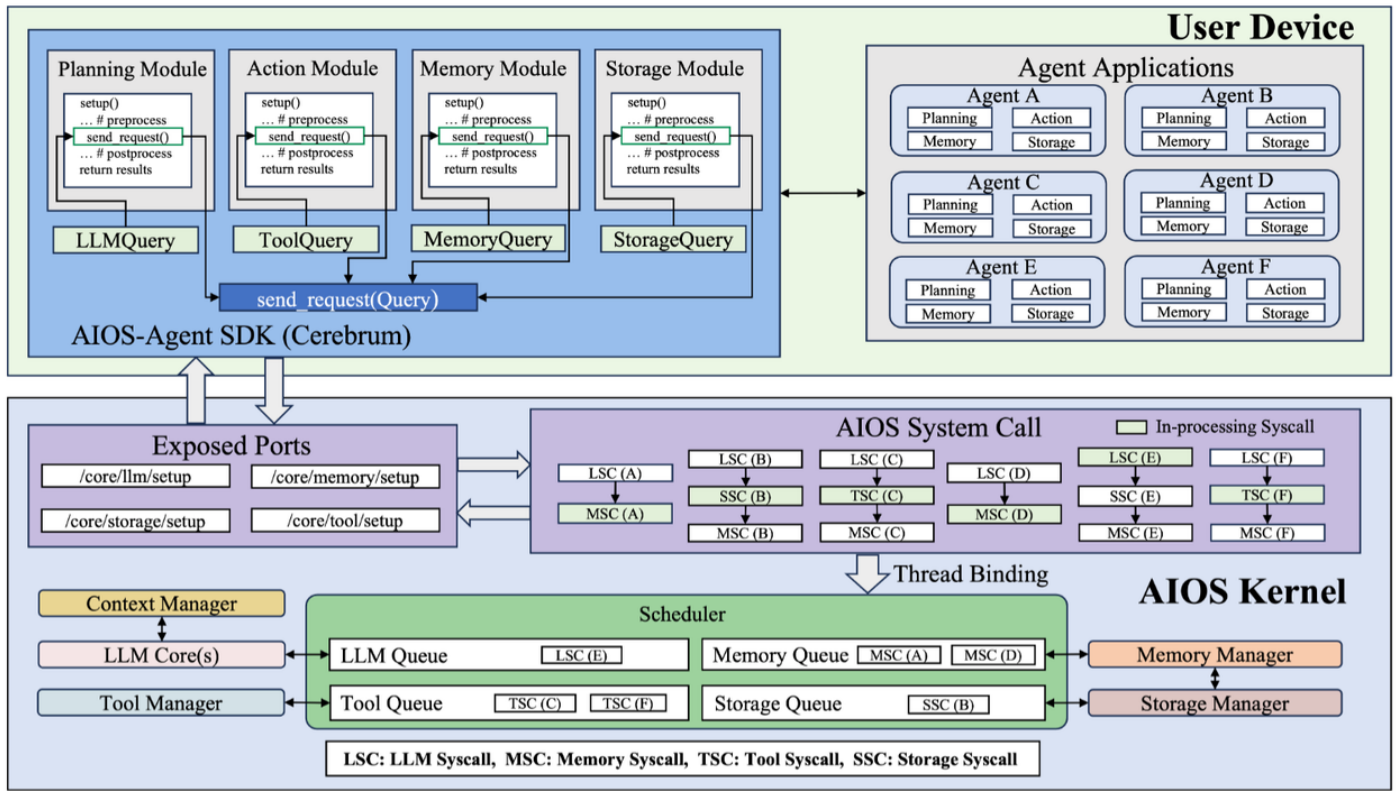


Figure 1: The architecture of Cerebrum on the basis of AIOS.

3.6 LiteCUA: Computer as MCP Server for Computer-Use Agent on AIOS

研究团队提出AIOS 1.0平台，其核心是通过Model Context Protocol (MCP) 服务器架构将计算机转化为语言模型可理解的语境化环境，解决语言模型对世界的理解与计算机界面结构间的语义脱节问题。基于该平台开发的轻量级计算机使用代理LiteCUA，在OSWorld 基准测试中取得 14.66% 的成功率，超越多个专用代理框架，表明为语言模型语境化计算机环境是开发更强大计算机使用代理及迈向能与数字系统交互的 AGI 的有前景方向。

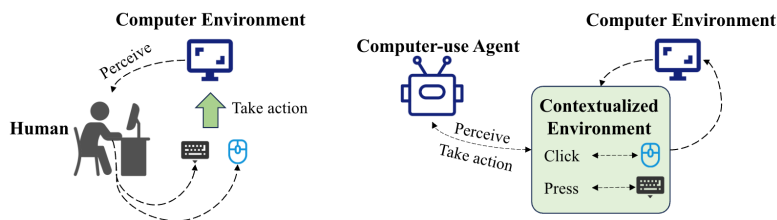


Figure 1: Illustration of the differences between how human operates computer and how computer-use agent (CUA) operates computer, where CUA requires a specific contextualized environment to understand the original computer environment and take actions.

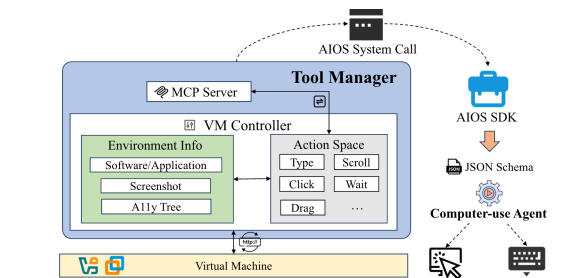


Figure 3: Pipeline for agents to take actions to interact with VM environments.

4. 代码演示&分析

4.1 运行&演示

4.1.1 修改配置

配置 aios/config/config.yaml 文件

代码块

```
1  api_keys:
2    deepseek: "xxxxx"    # deepseek API key
3
4
5  # LLM Configuration
6  llms:
7    models:
8      # DeepSeek API Models
9      - name: "deepseek/deepseek-chat"
10        backend: "deepseek"
```

添加deepseek adapter , 修改llm_core/adapter.py文件

代码块

```
1      api_providers = {
2          "openai": "OPENAI_API_KEY",
3          .....
4          "novita": "NOVITA_API_KEY",
5          "deepseek": "DEEPSEEK_API_KEY",
6      }
```

4.1.2 修改BUG

1. 安装出错：Issue: <https://github.com/agiresearch/AIOS/issues/501>

2. 代码的bug,aios/llm_core/utils.py:decode_litellm_tool_calls

第198行, if content is None or str(content).strip()=="":

